



Military Institute of Science and Technology

Department of Computer Science and Engineering

TERM PAPER

CSE 307: Operating Systems

Learned Disk Scheduler Selector

Track 2: Disk Scheduling

Submitted by	Najifa Raksanda
Student ID	202414010
Section	A
Submitted to	Khaled Hasan Irfan

Code and experimental results:

https://github.com/najifa-raksanda/0s_DiskScheduling

Learned Disk Scheduler Selector

Abstract

This project compares FCFS, SSTF, SCAN, and C-SCAN using synthetic disk-request workloads. A small decision-tree classifier selects a scheduler for each request window and provides a confidence score. The experiment tests sequential, random, and bursty requests, followed by a timeline that changes from sequential to bursty requests. Performance is measured using total disk-head movement. The selector performs well on random requests, but it does not consistently outperform SSTF. Correct selections receive higher confidence on average, although the reliability plot shows that confidence is not perfectly calibrated.

1 Introduction

Disk scheduling determines the order in which pending disk requests are served. The order affects how far a disk head moves between requested tracks. FCFS follows arrival order, SSTF chooses the nearest pending request, and SCAN and C-SCAN serve requests through directional sweeps [1].

The aim of this project is to test whether request patterns can help a small classifier choose a scheduler with low head movement. It also checks whether the classifier's confidence is higher when its selection is correct.

This is a simulation of disk scheduling. Running it in Kali Linux under Oracle VirtualBox provides the execution environment; it does not measure the physical movement of the host computer's storage device.

2 Scheduler Implementation

The simulated disk contains tracks numbered 0 to 199. SCAN and C-SCAN initially move toward track 0, following the convention used in the course examples [1].

- **FCFS:** Serves requests in arrival order.
- **SSTF:** Serves the nearest pending request. Equal-distance ties follow the original request order.
- **SCAN:** Serves requests toward track 0 and, if requests remain on the other side, reaches 0 before reversing.
- **C-SCAN:** Serves requests toward track 0, then wraps to track 199 when necessary and continues in the same service direction. The wrap is counted.

Movement is counted until the final pending request is served. For a head path $h_0, h_1, \dots, h_m$, total movement is

$$D = \sum_{i=1}^m |h_i - h_{i-1}|.$$

Here, D is measured in tracks. It is a proxy for seek cost, not a measured seek time in milliseconds.

3 Learned Selector

Each window contains 24 requests. The classifier uses seven features: mean track position, track-position variance, mean adjacent jump, fraction of requests below the head, fraction of unique requests, mean distance from the head, and the current head position.

All four schedulers are evaluated on each training window. The scheduler with the lowest movement becomes the training label. Ties use the fixed order FCFS, SSTF, SCAN, then C-SCAN.

A scikit-learn decision tree is trained with maximum depth 5 and a minimum leaf size of 20 [2]. Its predicted class probability is used as the confidence score. During evaluation, a prediction is counted as correct whenever its movement equals the minimum, including tied minimum-cost schedulers.

4 Experimental Setup

The experiment was run in Kali Linux inside Oracle VirtualBox. Training uses 1,200 windows, with 400 windows from each workload type. Independent testing uses 150 windows per type. The training seed is 307 and the testing seed is 308.

The workloads are generated as follows:

- **Sequential:** Requests progress in small, fixed steps toward higher or lower tracks.
- **Random:** Requests are sampled uniformly across tracks 0–199.
- **Bursty:** Requests concentrate around one hot region, with occasional uniformly random requests.

The term “bursty” here describes spatial clustering. The simulator does not model request-arrival times.

A separate 100-window timeline uses sequential requests for the first 50 windows and bursty requests for the remaining 50. The selected scheduler’s final head position becomes the starting head for the next window. Within each window, every scheduler receives the same requests and starting head. The classifier remains fixed throughout the timeline.

Consequently, the timeline compares alternative decisions from a shared state. Its baseline totals are not independent continuous runs with each scheduler carrying its own head.

5 Results and Discussion

Table 1 reports total movement for the independent test workloads. “Selected” means the classifier’s choice. “Oracle” is the minimum movement available from the four schedulers in each window.

Table 1: Total head movement in tracks: 150 windows per workload.

Policy	Sequential	Random	Bursty
FCFS	16,433	245,489	82,670
SSTF	14,114	39,425	25,207
SCAN	25,063	42,445	34,821
C-SCAN	38,511	57,073	48,809
Selected	16,157	38,087	25,470
Oracle	14,085	37,304	24,863

The selector’s accuracy is 65.33% for sequential requests, 76.67% for random requests, and 89.33% for bursty requests. For random requests, it achieves lower total movement than any single fixed scheduler. However, SSTF performs better than the selector on sequential and bursty requests.

Figure 1 shows the workload-change experiment. Selector accuracy increases from 74% before the change to 90% afterward. This does not demonstrate online learning: the model is unchanged, and the workload becomes different.

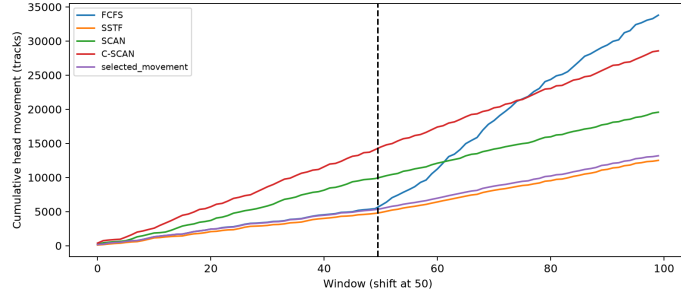


Figure 1: Cumulative head movement during the timeline. The dashed line marks the change from sequential to bursty requests at window 50.

6 Confidence and Calibration

Across all 550 evaluation windows, average confidence is 85.79% for correct selections and 62.34% for incorrect selections. Thus, the classifier generally assigns higher confidence to correct choices.

However, higher confidence on correct predictions is not enough to establish calibration. A calibrated confidence score should approximately match the observed correctness rate of predictions with similar confidence [3].

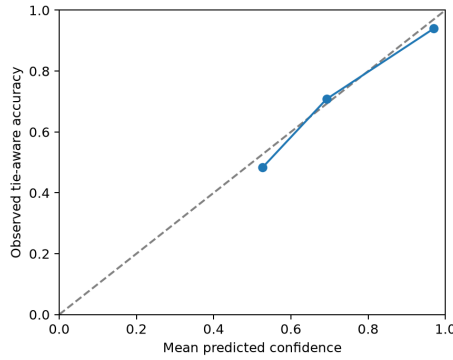


Figure 2: Mean predicted confidence versus observed accuracy in confidence bins. The dashed diagonal represents perfect calibration.

Figure 2 shows that the scores do not match accuracy exactly. In the highest confidence bin, mean confidence is approximately 96.97%, while observed accuracy is 93.95%. This indicates some overconfidence. The confidence scores are useful indicators, but they should not be interpreted as perfectly reliable probabilities.

7 Limitations

The experiment measures head movement rather than real disk latency, throughput, or virtual-machine I/O performance. It does not include rotational delay, timed arrivals, waiting-time fairness, or starvation measurements.

Only one training/testing seed pair is used, so the results do not establish performance across all possible workloads. C-SCAN receives no winning training labels in this dataset, which also limits the selector’s learned choices.

SSTF is a strong baseline in these experiments. The results therefore support workload-dependent selection, but do not show that the classifier always improves performance.

8 Conclusion

The project implements all four required schedulers and evaluates a learned selector on matching request windows. The selector performs particularly well on random requests, while SSTF remains better on two of the three independent workload types. The shift experiment demonstrates how performance changes when the request pattern changes. Confidence is higher for correct selections on average, but the calibration plot reveals some overconfidence.

AI Assistance Disclosure

I used ChatGPT/Codex for guidance on the project steps, help with coding, and assistance with documentation. I ran the experiment in Kali Linux using Oracle VirtualBox.

References

- [1] Khaled Hasan Irfan, *CSE 307 Operating Systems: Disk Scheduling Lecture Slides*, course material.
- [2] scikit-learn developers, *Decision Trees*. Available: <https://scikit-learn.org/stable/modules/tree.html>
- [3] scikit-learn developers, *Probability Calibration*. Available: <https://scikit-learn.org/stable/modules/calibration.html>